PRODUCT REQUIREMENTS DOCUMENT

MedFlow AI

Multi-Agent Clinical Encounter Orchestrator

Powered by Google Gemini • LangChain • LangGraph • ChromaDB RAG

Document ID	PRD-MEDFLOW-2026-001
Version	1.0
Status	Draft — Ready for Engineering Review
Author	[Your Name]
Date	March 4, 2026
Classification	Internal — Confidential

Revision History

Version	Date	Author	Changes
1.0	2026-03-04	[Your Name]	Initial PRD creation with full specification
0.9	2026-03-02	[Your Name]	Architecture review draft
0.1	2026-03-01	[Your Name]	Initial outline and scope definition

1. Executive Summary

1.1 Product Vision

MedFlow AI is a multi-agent clinical encounter orchestrator that transforms the patient intake-to-clinical-brief workflow using coordinated AI agents. The system conducts adaptive patient interviews, screens for drug interaction dangers via parallel RAG-powered evidence retrieval, classifies urgency using emergency severity indexing, generates structured clinical documentation, and queues all recommendations for mandatory clinician approval before any output reaches the patient.

This is a portfolio-grade demonstration project that showcases production-level AI engineering patterns: multi-agent orchestration, retrieval-augmented generation with ChromaDB, human-in-the-loop safety gates, real-time streaming, and full observability via LangSmith. It is explicitly designed as an educational and demonstration tool, not for actual clinical use.

1.2 Problem Statement

Clinical intake workflows today involve manual data collection, fragmented information systems, and cognitive overload for providers. An average emergency department physician makes over 10,000 cognitive decisions per shift, yet critical information like drug interactions is often caught late or missed entirely. Meanwhile, AI engineering candidates struggle to demonstrate production-grade agent architectures in portfolio projects, defaulting to simple chatbot demos that fail to impress senior hiring managers.

MedFlow AI solves both problems: it demonstrates a genuinely complex, healthcare-relevant multi-agent architecture while providing a realistic clinical workflow that showcases every major LangChain and LangGraph pattern in an organic, non-contrived manner.

1.3 Success Criteria

Metric	Target	Measurement Method
End-to-end encounter completion	< 90 seconds (demo scenario)	LangSmith trace timing
Drug interaction detection accuracy	> 95% on test set of 20 known pairs	Automated eval suite
RAG retrieval relevance	> 0.85 cosine similarity on top-3 chunks	ChromaDB similarity scores
Concept coverage	35/35 LangChain + LangGraph concepts	Architecture mapping audit
Streaming latency (first token)	< 500ms	Frontend performance logging
Human-in-the-loop gate	100% of outputs require approval	Graph topology enforcement
LangSmith trace completeness	100% of agent decisions traced	LangSmith dashboard audit

1.4 Stakeholders

Role	Name / Team	Responsibility
Product Owner	[Your Name]	PRD authorship, architecture decisions, demo delivery
AI Engineer	[Your Name]	Full-stack implementation: backend, RAG, frontend

Hiring Manager	Target interviewer	Portfolio evaluation, technical deep-dive
End User (Persona)	Dr. Sarah Chen, ED Physician	Provider reviewing AI-generated clinical briefs
End User (Persona)	Alex Rivera, Patient	Patient interacting with intake chatbot

2. Strategic Context

2.1 Market Opportunity

The healthcare AI market reached an estimated $32.2 billion in 2025, with clinical decision support and ambient documentation representing the fastest-growing segments. Multi-agent AI systems for healthcare are at the forefront of this wave, with Microsoft launching its Healthcare Agent Orchestrator at Build 2025 and Oxford University Hospitals deploying TrustedMDT for multi-disciplinary tumor boards.

For AI engineering job seekers, demonstrating healthcare domain expertise combined with production-grade agent architecture positions candidates for roles at companies like Abridge, Hippocratic AI, Microsoft Health, Google Health, and numerous health-tech startups that are actively hiring LLM engineers.

2.2 Competitive Landscape (Portfolio Projects)

Project Type	Limitations	MedFlow Advantage
Simple RAG chatbot	No agent orchestration, no HITL, no streaming	Full multi-agent graph with 6 specialists
Single-agent assistant	No parallelism, no sub-graphs, no state mgmt	Fan-out/fan-in, map-reduce, typed state
Todo app with AI	Trivial domain, no real-world complexity	Healthcare domain with genuine safety requirements
LangChain tutorial clone	Looks copied, no original architecture	Original architecture with organic concept integration

2.3 Technical Differentiation

- **Gemini-Native:** Built on Google Gemini (via Groq for speed), leveraging multimodal capabilities for medical image understanding and long-context processing.
- **RAG-First Evidence:** ChromaDB-powered retrieval-augmented generation over medical literature, drug databases, and clinical guidelines — not just LLM hallucination.
- **True Multi-Agent:** Six specialist sub-graphs coordinated by a supervisor, not a single monolithic prompt.
- **Safety by Architecture:** Human-in-the-loop is not a feature flag — it is structurally impossible for AI recommendations to reach the patient without clinician approval.

3. Technical Architecture

3.1 Technology Stack

Layer	Technology	Rationale
LLM Provider	Google Gemini 2.0 Flash (via langchain-google-genai) or Groq (Llama 3.3 70B)	Free tier, multimodal, fast inference; Groq for ultra-low-latency alternative
Embeddings	GoogleGenerativeAIEmbeddings (models/gemini-embedding-exp-03-07) or sentence-transformers	Native Gemini embeddings for semantic coherence; fallback to HuggingFace for offline
Agent Framework	LangGraph 0.3+	Stateful graph orchestration with cycles, breakpoints, and persistence
Chain/Tool Layer	LangChain 0.3+	Provider-agnostic model abstraction, tool binding, message schemas
Vector Database	ChromaDB (persistent mode)	Lightweight, embedded, zero-infra RAG; perfect for portfolio demo
Checkpointer	SqliteSaver (dev) / PostgresSaver (prod)	Graph state persistence across sessions
Long-Term Memory	LangGraph InMemoryStore	Cross-session patient profile and encounter history
Observability	LangSmith	Full trace logging, evaluation datasets, latency monitoring
Frontend	Next.js + React + Tailwind CSS	Modern SSR framework with streaming support
Deployment	Docker + LangGraph CLI (langgraph dev)	Local development server; optional cloud deploy

3.2 System Architecture Overview

The system follows a supervisor-specialist pattern implemented as a LangGraph StateGraph. The top-level graph routes patient encounters through six specialist sub-graphs, with parallel execution, conditional routing, and mandatory human gates.

Architecture Flow

```
Patient → React Chat UI (streaming SSE)
                  │
     ┌─── Supervisor Agent (LangGraph StateGraph) ───┐
     │                                                │
     │  [1] Intake Agent (conversational sub-graph)   │
     │       │                                        │
     │       ├── Multimodal input (text + med photos) │
     │       └── Adaptive clinical interview          │
     │                                                │
     │  [2] PARALLEL FAN-OUT:                         │
     │       ├── Safety Sentinel (map-reduce drugs)   │
     │       ├── Evidence Synthesizer (RAG + ReAct)   │
     │       └── Patient Education (dynamic config)   │
     │                                                │
     │  [3] FAN-IN → Triage Router (conditional edges) │
```

```
    │                                                    │
    │  [4] Clinical Brief Generator (chain pattern)      │
    │                                                    │
    │  [5] HUMAN-IN-THE-LOOP (breakpoint + interrupt)    │
    │       Provider reviews, edits, approves            │
    │                                                    │
    └────────────────────────────────────────────────────┘
              │
   ChromaDB (RAG)  │  LangGraph Store (memory)  │  LangSmith (traces)
```

3.3 State Schema Design

3.3.1 Primary Encounter State

```
from typing import TypedDict, Annotated
from langgraph.graph import add_messages
from pydantic import BaseModel, Field

class Medication(BaseModel):
    name: str = Field(description="Drug name")
    dose: str = Field(description="Dosage e.g. 500mg")
    frequency: str = Field(description="e.g. BID, daily")
    route: str = Field(default="oral")

class SafetyAlert(BaseModel):
    severity: str  # "critical" | "moderate" | "low"
    drug_pair: tuple[str, str]
    description: str
    source: str
    evidence_score: float

class EncounterState(TypedDict):
    # Core conversation
    messages: Annotated[list, add_messages]

    # Patient data (populated by Intake Agent)
    patient_id: str
    chief_complaint: str
    symptoms: list[str]
    medications: list[Medication]
    allergies: list[str]
    vitals: dict
    medical_history: str

    # Safety analysis (populated by Safety Sentinel)
    safety_alerts: Annotated[list[SafetyAlert], safety_reducer]

    # Evidence (populated by Evidence Synthesizer via RAG)
    rag_context: list[dict]  # [{source, content, score}]
    evidence_summary: str

    # Triage (populated by Router)
    triage_level: str  # "ESI-1" through "ESI-5"
    triage_reasoning: str

    # Documentation (populated by Brief Generator)
    clinical_brief: dict  # SOAP note structure
    patient_instructions: str
```

```
    # Workflow control
    status: str  # "intake"|"analyzing"|"reviewing"|"complete"
    current_agent: str
    attempt_count: int
```

3.3.2 Input / Output Schemas (API Boundary)

```
class InputState(TypedDict):
    messages: Annotated[list, add_messages]
    patient_id: str

class OutputState(TypedDict):
    clinical_brief: dict
    triage_level: str
    safety_alerts: list[SafetyAlert]
    patient_instructions: str
    status: str

# Internal state includes all fields; callers only see Input/Output
graph = StateGraph(
    EncounterState,
    input=InputState,
    output=OutputState
)
```

3.3.3 Custom State Reducers

```
def safety_reducer(existing: list[SafetyAlert], new: list[SafetyAlert]) ->
list[SafetyAlert]:
    """Accumulates safety alerts from parallel drug checks, deduplicates by drug
pair."""
    seen = {(a.drug_pair[0], a.drug_pair[1]) for a in existing}
    merged = list(existing)
    for alert in new:
        key = (alert.drug_pair[0], alert.drug_pair[1])
        if key not in seen:
            merged.append(alert)
            seen.add(key)
    return sorted(merged, key=lambda a: {"critical": 0, "moderate": 1, "low": 2}
[a.severity])
```

3.4 RAG Architecture (ChromaDB)

The Evidence Synthesizer agent uses retrieval-augmented generation over three ChromaDB collections to ground its responses in verified medical knowledge rather than relying on LLM parametric memory.

3.4.1 ChromaDB Collections

Collection	Content Source	Chunking Strategy
drug_interactions	openFDA adverse event reports, FDA drug labels, RxNorm data (pre-indexed)	1000 chars, 200 overlap, metadata: {drug_name, severity, source_date}
clinical_guidelines	PubMed abstracts (top 500 emergency medicine), WHO ICD-11 descriptions	800 chars, 150 overlap, metadata: {condition, publication_year, journal}
patient_education	MedlinePlus consumer health articles, CDC fact sheets (public domain)	600 chars, 100 overlap, metadata: {topic, reading_level, language}

3.4.2 RAG Pipeline Implementation

```
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

# ── Embedding Model ──
embeddings = GoogleGenerativeAIEmbeddings(
    model="models/gemini-embedding-exp-03-07",
    google_api_key=os.getenv("GOOGLE_API_KEY")
)

# ── ChromaDB Collections ──
drug_store = Chroma(
    collection_name="drug_interactions",
    persist_directory="./chroma_db",
    embedding_function=embeddings,
)

guidelines_store = Chroma(
    collection_name="clinical_guidelines",
    persist_directory="./chroma_db",
    embedding_function=embeddings,
)

education_store = Chroma(
    collection_name="patient_education",
    persist_directory="./chroma_db",
    embedding_function=embeddings,
)

# ── Retriever with score threshold ──
drug_retriever = drug_store.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"k": 5, "score_threshold": 0.75}
)

guidelines_retriever = guidelines_store.as_retriever(
    search_type="mmr",  # Maximal Marginal Relevance for diversity
    search_kwargs={"k": 4, "fetch_k": 10}
)
```

3.4.3 Data Ingestion Pipeline

```
# scripts/ingest_medical_data.py
from langchain_community.document_loaders import PyPDFDirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

def ingest_drug_data():
    """Fetch FDA drug labels and adverse events, chunk and embed into ChromaDB."""
    import requests

    # Fetch from openFDA API
    url = "https://api.fda.gov/drug/label.json"
    params = {"limit": 100, "search": "warnings:interaction"}
    response = requests.get(url, params=params)
    results = response.json().get("results", [])
```

```
    docs = []
    for r in results:
        text = " ".join(r.get("warnings", [""]))
        docs.append(Document(
            page_content=text,
            metadata={"drug": r.get("openfda", {}).get("brand_name", ["Unknown"])
[0],
                      "source": "openFDA"}
        ))

    splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
    chunks = splitter.split_documents(docs)

    drug_store.add_documents(chunks)
    print(f"Ingested {len(chunks)} drug interaction chunks")

def ingest_pubmed_abstracts(query, max_results=200):
    """Fetch PubMed abstracts via E-utilities and index into ChromaDB."""
    import requests

    # Step 1: Search PubMed
    search_url = "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi"
    search_params = {"db": "pubmed", "term": query, "retmax": max_results,
"retmode": "json"}
    ids = requests.get(search_url, params=search_params).json()
    pmids = ids["esearchresult"]["idlist"]

    # Step 2: Fetch abstracts
    fetch_url = "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi"
    fetch_params = {"db": "pubmed", "id": ",".join(pmids), "rettype": "abstract",
"retmode": "text"}
    abstracts = requests.get(fetch_url, params=fetch_params).text

    # Step 3: Chunk and embed
    splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=150)
    docs = [Document(page_content=abstracts, metadata={"source": "PubMed", "query":
query})]
    chunks = splitter.split_documents(docs)
    guidelines_store.add_documents(chunks)
```

3.5 LLM Configuration

3.5.1 Gemini Configuration (Primary)

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash",
    temperature=0.1,          # Low temp for clinical accuracy
    max_retries=2,
    google_api_key=os.getenv("GOOGLE_API_KEY"),
)

# For multimodal (medication photo analysis)
vision_llm = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash",
    temperature=0,
)
```

3.5.2 Groq Configuration (Low-Latency Alternative)

```
from langchain_groq import ChatGroq

llm = ChatGroq(
    model="llama-3.3-70b-versatile",
    temperature=0.1,
    groq_api_key=os.getenv("GROQ_API_KEY"),
)

# Config-driven model selection
def get_llm(config):
    provider = config.get("configurable", {}).get("llm_provider", "gemini")
    if provider == "groq":
        return ChatGroq(model="llama-3.3-70b-versatile", temperature=0.1)
    return ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0.1)
```

4. Agent Specifications

Each agent is implemented as a self-contained LangGraph sub-graph with its own internal state, nodes, and edges. The Supervisor composes them into the top-level encounter graph.

4.1 Agent 1: Intake Agent

Property	Specification
Pattern	Conversational chain with adaptive questioning
Sub-graph Nodes	greet → collect_complaint → collect_symptoms → collect_medications → collect_history → summarize
Tools	analyze_image (multimodal medication photo OCR)
Memory	Short-term: full message history within thread via checkpointer
Input	Patient messages (text + optional images)
Output	Structured patient data: chief_complaint, symptoms, medications, allergies, vitals
LangChain Concepts	Foundational models, multimodal messages, short-term memory, tool binding
LangGraph Concepts	Sub-graph, chain pattern, state schema, persistent memory (checkpointer)

Key Implementation: Multimodal Medication Extraction

```
from langchain_core.messages import HumanMessage
import base64

@tool
def analyze_medication_image(image_base64: str) -> str:
    """Extract medication names and dosages from a photo of pill bottles."""
    message = HumanMessage(content=[
        {"type": "image_url",
         "image_url": {"url": f"{image_base64}"}},
        {"type": "text",
         "text": "List every medication visible in this image. For each, provide:
drug name, dosage, and frequency. Return as JSON array."}
    ])
    response = vision_llm.invoke([message])
    return response.content
```

4.2 Agent 2: Safety Sentinel

Property	Specification
Pattern	Map-reduce with parallel RAG lookups
Sub-graph Nodes	extract_pairs → [Send() per pair] check_interaction → compile_safety_report
Tools	query_drug_interactions (ChromaDB retriever), query_fda_adverse_events (openFDA API)
RAG Collections	drug_interactions (ChromaDB)
Input	List of Medication objects from Intake
Output	List of SafetyAlert objects with severity ranking
Trigger	Fires in parallel with Evidence Synthesizer after Intake completes

LangChain Concepts	Custom tools, tool binding
LangGraph Concepts	Map-reduce (Send() API), parallelisation, custom state reducers, sub-graph

Key Implementation: Map-Reduce Drug Interaction Check

```
from langgraph.constants import Send
from itertools import combinations

def generate_drug_pairs(state: EncounterState):
    """Fan-out: create a Send() for each medication pair."""
    meds = state["medications"]
    pairs = list(combinations(meds, 2))
    return [
        Send("check_single_interaction", {
            "drug_a": pair[0].name,
            "drug_b": pair[1].name,
            "safety_alerts": []
        })
        for pair in pairs
    ]

def check_single_interaction(state):
    """Check one drug pair against ChromaDB + openFDA."""
    query = f"{state["drug_a"]} interaction with {state["drug_b"]}"

    # RAG lookup from ChromaDB
    rag_results = drug_retriever.invoke(query)

    # LLM analysis with retrieved context
    context = "\n".join([doc.page_content for doc in rag_results])
    prompt = f"""Based on this evidence:\n{context}\n
    Assess the interaction between {state["drug_a"]} and {state["drug_b"]}.
    Respond with JSON: {{"severity": "critical|moderate|low|none", "description":
"..."}}"""

    result = llm.invoke(prompt)
    # Parse and return SafetyAlert
    return {"safety_alerts": [parsed_alert]}
```

4.3 Agent 3: Evidence Synthesizer

Property	Specification
Pattern	ReAct agent with iterative RAG retrieval
Sub-graph Nodes	agent_node (LLM + tools) ↔ tool_node (execute tools) [cycle]
Tools	search_clinical_guidelines (ChromaDB retriever), search_pubmed_live (PubMed E-utilities API), retrieve_drug_labels (openFDA API)
RAG Collections	clinical_guidelines (ChromaDB)
Max Iterations	3 search-evaluate-refine cycles
Input	Chief complaint, symptoms, medications, medical history
Output	evidence_summary (synthesized text), rag_context (source citations with scores)
LangChain Concepts	ReAct pattern, tool-calling loop, RAG retrieval
LangGraph Concepts	Conditional edges (cycles), agent pattern, state management

Key Implementation: RAG-Powered ReAct Agent

```
@tool
def search_clinical_guidelines(query: str) -> str:
    """Search ChromaDB clinical guidelines for relevant evidence."""
    results = guidelines_retriever.invoke(query)
    formatted = []
    for doc in results:
        formatted.append({
            "content": doc.page_content,
            "source": doc.metadata.get("source", "unknown"),
            "relevance": doc.metadata.get("score", 0)
        })
    return json.dumps(formatted)

@tool
def search_pubmed_live(query: str, max_results: int = 5) -> str:
    """Search PubMed for recent medical literature via E-utilities API."""
    import requests
    search_url = "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi"
    params = {"db": "pubmed", "term": query, "retmax": max_results, "retmode":
"json"}
    ids = requests.get(search_url, params=params).json()
    pmids = ids["esearchresult"]["idlist"]

    fetch_url = "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi"
    fetch_params = {"db": "pubmed", "id": ",".join(pmids), "rettype": "abstract",
"retmode": "text"}
    return requests.get(fetch_url, params=fetch_params).text

# Bind tools to Gemini
evidence_llm = llm.bind_tools([search_clinical_guidelines, search_pubmed_live])

# ReAct loop via LangGraph
def should_continue(state):
    last_msg = state["messages"][-1]
    if hasattr(last_msg, "tool_calls") and last_msg.tool_calls:
        return "tools"
    return "end"
```

4.4 Agent 4: Triage Router

Property	Specification
Pattern	Router with conditional edges
Logic	ESI (Emergency Severity Index) 5-level classification via structured LLM output
Routing	ESI-1/2 → fast_track_brief, ESI-3 → full_workup, ESI-4/5 → standard_brief
Input	Full encounter state (symptoms, vitals, safety alerts, evidence)
Output	triage_level, triage_reasoning + routing decision
LangGraph Concepts	Router pattern, conditional edges (add_conditional_edges), structured output

Key Implementation: Conditional Routing

```
def triage_router(state: EncounterState) -> str:
    """Route based on ESI level determined by triage node."""
    level = state["triage_level"]
```

```
    if level in ("ESI-1", "ESI-2"):
        return "fast_track_brief"
    elif level == "ESI-3":
        return "full_workup"
    else:
        return "standard_brief"

graph.add_conditional_edges("triage", triage_router, {
    "fast_track_brief": "generate_brief",
    "full_workup": "generate_brief",
    "standard_brief": "generate_brief",
})
```

4.5 Agent 5: Clinical Brief Generator

Property	Specification
Pattern	Linear chain (no branching)
Sub-graph Nodes	collect_inputs → generate_soap → validate_completeness → format_output
Output Format	SOAP Note: Subjective, Objective, Assessment (with differentials), Plan
Input	Full encounter state including safety alerts, evidence, and triage
LangGraph Concepts	Chain pattern, state schema

4.6 Agent 6: Patient Education Agent

Property	Specification
Pattern	Dynamic agent (runtime-configurable)
RAG Collection	patient_education (ChromaDB)
Dynamic Config	language (en/es/bn), reading_level (6th_grade/8th_grade/college), care_setting (ed/primary/pediatrics)
Output	Plain-language patient instructions with medication explanations and warning signs
LangChain Concepts	Dynamic agents (configurable fields), RAG retrieval
LangGraph Concepts	Assistants pattern (multi-tenant), sub-graph

Key Implementation: Dynamic Configuration

```
def patient_education_node(state, config):
    """Generate patient instructions based on runtime configuration."""
    settings = config.get("configurable", {})
    reading_level = settings.get("reading_level", "8th_grade")
    language = settings.get("language", "en")
    care_setting = settings.get("care_setting", "ed")

    # RAG: retrieve relevant patient education materials
    query = state["chief_complaint"] + " patient education"
    edu_docs = education_store.similarity_search(query, k=3)
    context = "\n".join([d.page_content for d in edu_docs])

    prompt = f"""Using this reference material:\n{context}\n
    Generate patient discharge instructions for: {state['chief_complaint']}
    Reading level: {reading_level}
    Language: {language}
```

```
    Care setting: {care_setting}
    Include: medication explanations, warning signs, follow-up timeline."""

    result = llm.invoke(prompt)
    return {"patient_instructions": result.content}
```

5. Human-in-the-Loop Specification

5.1 Static Breakpoints

Every encounter pauses at the provider review node before any AI-generated content is finalized. This is a structural guarantee, not a runtime check.

```
app = graph.compile(
    checkpointer=SqliteSaver.from_conn_string("encounters.db"),
    interrupt_before=["provider_review"]  # Always pause here
)
```

5.2 Dynamic Breakpoints (interrupt)

Critical safety findings trigger immediate workflow interruption regardless of which agent is currently active.

```
from langgraph.types import interrupt

def safety_gate(state: EncounterState):
    """Dynamic interrupt for critical drug interactions."""
    critical_alerts = [a for a in state["safety_alerts"] if a.severity ==
"critical"]
    if critical_alerts:
        decision = interrupt({
            "type": "CRITICAL_SAFETY_ALERT",
            "alerts": [a.dict() for a in critical_alerts],
            "message": f"URGENT: {len(critical_alerts)} critical drug
interaction(s) detected. Provider review required immediately.",
            "actions": ["acknowledge_and_continue", "halt_encounter",
"override_with_reason"]
        })
        if decision["action"] == "halt_encounter":
            return {"status": "halted_by_provider"}
        elif decision["action"] == "override_with_reason":
            return {"status": "override", "override_reason": decision["reason"]}
    return {"status": "analyzing"}
```

5.3 State Editing (Provider Feedback)

At the review breakpoint, the provider can modify any field in the encounter state before resuming.

Editable Field	Edit Type	Example
triage_level	Override classification	Upgrade ESI-4 → ESI-3 based on clinical judgment
safety_alerts	Dismiss false positive	Remove low-severity alert for known safe combination
clinical_brief.plan	Add/modify orders	Add "Order D-dimer if PE suspected"
medications	Correct medication list	Fix dosage from 500mg to 250mg
patient_instructions	Edit discharge text	Add specific follow-up date

```
# Provider edits state at breakpoint
app.update_state(
    config,
    values={"triage_level": "ESI-3", "clinical_brief": edited_brief},
    as_node="provider_review"
```

```
)

# Resume execution with corrected state
for event in app.stream(None, config, stream_mode="updates"):
    print(event)
```

5.4 Time Travel (Audit & QA)

Every node execution creates a checkpoint. Quality reviewers can rewind to any point in the encounter to audit AI decision-making.

```
# Browse full encounter history
history = list(app.get_state_history(config))
for checkpoint in history:
    print(f"Step: {checkpoint.metadata.get('step')},
           Node: {checkpoint.metadata.get('source')}")
    print(f"  Triage: {checkpoint.values.get('triage_level')}")
    print(f"  Alerts: {len(checkpoint.values.get('safety_alerts', []))}")

# Rewind to before triage and re-run with different vitals
target = history[4]  # Before triage node
app.update_state(
    {**config, "configurable": {**config["configurable"], "checkpoint_id":
target.config["configurable"]["checkpoint_id"]}},
    values={"vitals": {"bp": "180/110", "hr": 110}}
)
```

6. Memory Architecture

6.1 Short-Term Memory (Per Encounter)

Mechanism	Implementation	Purpose
Message History	Annotated[list, add_messages] + SqliteSaver checkpointer	Full conversation context within the current encounter thread
Message Trimming	trim_messages(max_tokens=4000, strategy="last", include_system=True)	Keep context window bounded during long intake interviews
Message Filtering	Filter out tool_call / tool_result messages before Brief Generator	Agent-to-agent coordination hidden from synthesis steps

6.2 Long-Term Memory (Cross-Encounter)

LangGraph Store provides persistent cross-session memory using namespace-scoped key-value storage.

6.2.1 Profile Memory Pattern

A single, continuously updated patient profile document. New facts merge with existing data.

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

# Write profile after intake
def save_patient_profile(state, config, *, store):
    patient_id = config["configurable"]["patient_id"]
    namespace = ("patients", patient_id, "profile")

    existing = store.get(namespace, "demographics")
    profile = existing.value if existing else {}

    # Merge new data
    profile["allergies"] = state.get("allergies", profile.get("allergies", []))
    profile["chronic_conditions"] = state.get("medical_history", "")
    profile["updated_at"] = datetime.now().isoformat()

    store.put(namespace, "demographics", profile)
    return state
```

6.2.2 Collection Memory Pattern

Append-only log of past encounters. Each encounter adds a new item without overwriting previous ones.

```
def save_encounter_summary(state, config, *, store):
    patient_id = config["configurable"]["patient_id"]
    namespace = ("patients", patient_id, "encounters")
    encounter_id = f"enc_{datetime.now().strftime("%Y%m%d_%H%M%S")}"

    store.put(namespace, encounter_id, {
        "date": datetime.now().isoformat(),
        "chief_complaint": state["chief_complaint"],
        "triage_level": state["triage_level"],
        "diagnosis": state["clinical_brief"].get("assessment", ""),
        "medications_at_visit": [m.name for m in state["medications"]],
```

```
        "safety_alerts_count": len(state["safety_alerts"]),
    })
    return state
```

6.2.3 Conversation Summarization (Returning Patients)

```
def load_patient_context(state, config, *, store):
    """For returning patients, load profile + summarized encounter history."""
    patient_id = config["configurable"]["patient_id"]

    # Load profile
    profile = store.get(("patients", patient_id, "profile"), "demographics")

    # Load past encounters
    past_encounters = store.search(("patients", patient_id, "encounters"))

    # Summarize history into system message
    if past_encounters:
        history_text = "\n".join([
            f"- {e.value["date"][:10]}: {e.value["chief_complaint"]} (Triage:
{e.value["triage_level"]})"
            for e in past_encounters
        ])
        summary = llm.invoke(
            f"Summarize this patient\'s encounter history in 2-3 sentences:\
n{history_text}"
        )
        return {"medical_history": summary.content}
    return state
```

7. Middleware & Safety Layer

7.1 Middleware Pipeline

Three middleware layers run as pre/post-processing nodes at the graph boundaries, enforcing cross-cutting safety concerns without polluting individual agent logic.

Layer	Position	Implementation
PII Redaction	Pre-processing (before Intake)	Regex-based scrubbing of SSN patterns (XXX-XX-XXXX), phone numbers, email addresses, and exact dates of birth. Replaced with [REDACTED_SSN], [REDACTED_PHONE], etc.
Input Guardrails	Pre-processing (before Intake)	LLM classifier that rejects attempts to extract prescriptions, obtain controlled substance information, or bypass safety gates. Returns standardized refusal message.
Output Disclaimer	Post-processing (before response)	Appends medical disclaimer to every patient-facing output: "AI-generated content for educational demonstration. Not medical advice. Consult a healthcare professional."

7.1.1 PII Redaction Implementation

```
import re

PII_PATTERNS = {
    "ssn": (r"\b\d{3}-\d{2}-\d{4}\b", "[REDACTED_SSN]"),
    "phone": (r"\b\d{3}[-.]?\d{3}[-.]?\d{4}\b", "[REDACTED_PHONE]"),
    "email": (r"\b[\w.-]+@[\w.-]+\.\w+\b", "[REDACTED_EMAIL]"),
    "dob": (r"\b(?:0[1-9]|1[0-2])/(?:0[1-9]|[12]\d|3[01])/(?:19|20)\d{2}\b",
"[REDACTED_DOB]"),
}

def pii_redaction_node(state: EncounterState):
    """Middleware: scrub PII from the latest user message."""
    last_msg = state["messages"][-1]
    if hasattr(last_msg, "content") and isinstance(last_msg.content, str):
        cleaned = last_msg.content
        for name, (pattern, replacement) in PII_PATTERNS.items():
            cleaned = re.sub(pattern, replacement, cleaned)
        if cleaned != last_msg.content:
            from langchain_core.messages import HumanMessage
            return {"messages": [HumanMessage(content=cleaned, id=last_msg.id)]}
    return state
```

7.2 Safety Architecture Principles

1. No Autonomous Clinical Decisions: Every AI recommendation passes through a human gate (breakpoint) before reaching the patient.
2. Fail-Safe Defaults: If the Safety Sentinel fails or times out, the encounter is flagged for manual review rather than proceeding.
3. Audit Trail: LangSmith traces + checkpoint history provide a complete audit of every AI decision for regulatory review.

4. Scope Limitation: The system explicitly refuses to prescribe medications, diagnose conditions, or provide treatment plans without provider approval.
5. Persistent Disclaimer: Every interaction includes a visible disclaimer that this is a demonstration tool, not a clinical system.

7.3 Emergency Detection

```
EMERGENCY_KEYWORDS = [
    "can't breathe", "chest pain", "stroke", "seizure",
    "unconscious", "severe bleeding", "overdose", "suicidal"
]

def emergency_detector(state):
    """If emergency keywords detected, bypass normal flow."""
    last_msg = state["messages"][-1].content.lower()
    if any(kw in last_msg for kw in EMERGENCY_KEYWORDS):
        return {
            "status": "emergency_detected",
            "messages": [AIMessage(content=
                "If you are experiencing a medical emergency, please call 911 (or
your local emergency number) immediately.
                This AI system cannot provide emergency medical care.")]
        }
    return state
```

8. Streaming & Frontend UX

8.1 Streaming Modes

Mode	Use Case	Implementation
stream_mode="messages"	Token-by-token chat responses during Intake	SSE to frontend, rendered as typing animation
stream_mode="updates"	Agent activity indicators ("Safety Sentinel analyzing...")	Status cards in sidebar update in real-time
stream_mode="values"	Structured data updates (triage card, SOAP note panels)	React state updates trigger re-render of dashboard components

8.1.1 Backend Streaming Endpoint

```
# FastAPI streaming endpoint
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

@app.post("/encounter/stream")
async def stream_encounter(request: EncounterRequest):
    config = {"configurable": {
        "thread_id": request.thread_id,
        "patient_id": request.patient_id,
    }}

    async def event_generator():
        async for event in agent.astream(
            {"messages": [("human", request.message)]},
            config,
            stream_mode=["messages", "updates"]
        ):
            yield f"data: {json.dumps(event)}\n\n"

    return StreamingResponse(event_generator(), media_type="text/event-stream")
```

8.2 Frontend Components

Component	Data Source	Behavior
Chat Panel	stream_mode="messages"	Token-by-token typing animation, message bubbles
Agent Activity Bar	stream_mode="updates"	Shows active agent name + spinner, e.g. "Safety Sentinel checking 15 drug pairs..."
Triage Card	state.triage_level	Color-coded ESI badge (red=1, orange=2, yellow=3, green=4, blue=5)
Safety Alerts Panel	state.safety_alerts	Accordion list with severity icons, expandable for evidence details
SOAP Note Viewer	state.clinical_brief	Tabbed view: Subjective | Objective | Assessment | Plan
Provider Review Modal	interrupt() payload	Approve / Edit / Reject buttons with

		inline text editing
Assistant Selector	config.care_setting	Dropdown: Emergency Dept | Primary Care | Pediatrics

8.3 Double-Texting Handling

If the patient sends a new message while agents are processing, the system uses the 'interrupt' strategy: cancel the current run and restart with the updated context. This matches patient intent — corrections and additions should be processed together, not separately.

```
# LangGraph Platform configuration
# langgraph.json
{
  "graphs": {
    "encounter": {
      "file": "src/graph.py:encounter_graph",
      "multitask_strategy": "interrupt"
    }
  }
}
```

9. Observability & Evaluation

9.1 LangSmith Integration

```
# .env configuration
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=<your-langsmith-key>
LANGCHAIN_PROJECT=medflow-ai
```

Every agent decision, tool call, RAG retrieval, and state transition is automatically traced. The demo includes a walkthrough of a LangSmith trace showing the complete encounter flow.

9.2 Evaluation Datasets

Dataset	Size	Metrics
Drug Interaction Test Set	20 known interaction pairs + 10 safe pairs	Precision, Recall, F1 for interaction detection
Triage Classification	15 clinical scenarios with ground-truth ESI levels	Accuracy, confusion matrix
RAG Retrieval Quality	25 clinical queries with relevant chunk IDs	MRR@5, Precision@3, cosine similarity
End-to-End Encounter	5 complete encounter scenarios	Completion time, concept coverage, output quality (LLM-as-judge)

10. Deployment & Infrastructure

10.1 Local Development

```
# Clone and setup
git clone https://github.com/[you]/medflow-ai.git
cd medflow-ai
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Ingest medical data into ChromaDB
python scripts/ingest_medical_data.py

# Start LangGraph dev server
langgraph dev

# Start frontend
cd frontend && npm install && npm run dev
```

10.2 Docker Deployment

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
RUN python scripts/ingest_medical_data.py
EXPOSE 8000
CMD ["langgraph", "dev", "--host", "0.0.0.0", "--port", "8000"]
```

10.3 Assistants Configuration

Three pre-configured assistants share the same graph but differ in system prompts, tool permissions, and runtime configuration.

Assistant	System Prompt Focus	Reading Level	Special Tools
ED Assistant	Rapid triage, emergency protocols, time-critical	College	All tools enabled
Primary Care	Preventive care, lifestyle, referral generation	8th grade	Exclude emergency triage
Pediatrics	Age-adjusted dosing, growth charts, parent-friendly	6th grade	Add pediatric_dosing_calculator

11. One-Week Build Plan

Day	Focus Area	Deliverables
Day 1	Foundation: State + Graph + RAG	EncounterState schema, top-level StateGraph with all nodes/edges, ChromaDB setup with data ingestion scripts, SqliteSaver checkpointer, LangSmith tracing enabled. Working terminal flow: hardcoded input → basic chain → output.
Day 2	Intake Agent + Safety Sentinel	Intake sub-graph with adaptive questioning, multimodal image tool for medication photos, Safety Sentinel sub-graph with Send() map-reduce for drug pairs, ChromaDB drug_interactions retriever, custom safety_reducer.
Day 3	Evidence Synthesizer + Triage	ReAct sub-graph with ChromaDB clinical_guidelines retriever + live PubMed tool, iterative search-evaluate-refine loop, Triage Router with conditional edges and ESI classification.
Day 4	Brief Generator + Education + Memory	SOAP note chain, Patient Education agent with dynamic config, LangGraph Store for profile + collection memory patterns, conversation summarization for returning patients, message trimming.
Day 5	HITL + Middleware + Streaming	Static breakpoints at provider_review, dynamic interrupt() for critical alerts, state editing flow, PII redaction middleware, input guardrails, output disclaimers, all three streaming modes.
Day 6	Frontend + Integration	React/Next.js chat UI with streaming, encounter dashboard, provider review modal, assistant selector, agent activity indicators. Double-texting configuration. Time travel debug view.
Day 7	Polish + Deploy + Document	Docker deployment, LangSmith eval datasets, README with architecture diagram, 3-minute demo video recording, cleanup and testing.

12. LangChain/LangGraph Concept Coverage Matrix

Complete mapping of all 35+ course concepts to their organic implementation within MedFlow. Every concept serves a genuine purpose in the clinical workflow.

LangChain Course Concepts

#	Concept	MedFlow Implementation
1	Foundational Chat Models	ChatGoogleGenerativeAI / ChatGroq with provider-agnostic abstraction; config-driven model swap
2	Tool Binding (@tool, bind_tools)	6 custom tools: analyze_medication_image, search_clinical_guidelines, search_pubmed_live, query_drug_interactions, lookup_icd10, get_patient_profile
3	Short-Term Memory	Message history via checkpointer within encounter threads; trim_messages for long intakes
4	Multimodal Messages	Medication bottle photo upload → Gemini vision extraction → structured Medication objects
5	Model Context Protocol (MCP)	MCP server wrapping FDA + PubMed APIs for standardized tool discovery
6	Context and State	EncounterState typed dict with 15+ fields; selective context injection per agent
7	Multi-Agent Systems	Supervisor + 6 specialist agents with handoff pattern
8	Middleware	PII redaction, input guardrails, output disclaimer injection
9	Managing Long Conversations	Trimming (token-based), summarization (returning patients), filtering (hide tool msgs)
10	Human-in-the-Loop	Static breakpoint at provider_review; dynamic interrupt() for critical safety alerts
11	Dynamic Agents	Patient Education agent reconfigures per care_setting, language, reading_level
12	Agent Chat UI	React frontend with streaming, agent indicators, structured panels, review modal

LangGraph Course Concepts

#	Concept	MedFlow Implementation
13	Simple Graph (nodes, edges)	Top-level encounter graph with 8 nodes and directed edges
14	LangSmith Studio	Full tracing of every decision; eval datasets for quality scoring
15	Chain Pattern	Brief Generator: collect → generate_soap → validate → format (linear, no branching)
16	Router Pattern	Triage Router: conditional edges route ESI-1/2 vs ESI-3 vs ESI-4/5 to different workflows
17	ReAct Agent Pattern	Evidence Synthesizer: LLM + tools in a cycle (search → evaluate → refine → search)
18	Persistent Memory (Checkpointer)	SqliteSaver persists encounter state; browser refresh resumes mid-conversation
19	State Schema (TypedDict)	EncounterState with typed fields; Pydantic models for

		Medication, SafetyAlert
20	State Reducers	Custom safety_reducer accumulates and deduplicates alerts from parallel checks
21	Multiple Schemas	InputState (minimal) / OutputState (clinical brief) / EncounterState (internal)
22	Trim & Filter Messages	Token-based trimming for long intakes; filter tool messages before Brief Generator
23	Summarizing Messages	Returning patients get prior encounters condensed into 2-3 sentence summary
24	Streaming	Three modes: messages (tokens), updates (agent activity), values (structured data)
25	Breakpoints (Static)	interrupt_before=["provider_review"] on every encounter
26	Dynamic Breakpoints (interrupt)	interrupt() fires on critical drug interaction detection
27	Editing State + Human Feedback	Provider edits triage, dismisses alerts, modifies plan at review breakpoint
28	Time Travel	Full checkpoint history; QA reviewer can rewind and inspect any decision point
29	Parallelization	Fan-out: Safety + Evidence + Education run simultaneously after Intake
30	Sub-graphs	Each of 6 agents is a self-contained sub-graph with internal nodes/edges
31	Map-Reduce (Send)	Safety Sentinel: Send() per drug pair → parallel check → reduce to SafetyReport
32	Long-Term Memory (Store)	LangGraph InMemoryStore with namespace: ("patients", patient_id, "profile"|"encounters")
33	Profile Memory Pattern	PatientProfile: demographics, allergies, conditions — updated in-place
34	Collection Memory Pattern	EncounterHistory: append-only log of past encounters
35	Deployment	Docker + langgraph.json + langgraph dev; optional LangGraph Platform deploy
36	Double Texting	multitask_strategy: "interrupt" — cancel stale run, restart with latest message
37	Assistants	3 variants: ED, Primary Care, Pediatrics — same graph, different config

13. Risks & Mitigations

Risk	Probability	Impact	Mitigation
Gemini API rate limits during demo	Medium	Demo fails mid-encounter	Pre-cache common responses; Groq as fallback provider; implement retry with exponential backoff
ChromaDB retrieval returns irrelevant chunks	Medium	Hallucinated safety alerts	Score threshold filtering (>0.75); multiple retrieval strategies (similarity + MMR); manual eval set
LLM hallucination on drug interactions	High	False safety alerts erode trust	RAG grounds every claim; LLM must cite source chunks; mandatory HITL review gate
Scope creep beyond 1 week	High	Incomplete demo	Strict daily deliverables; MVP-first approach; defer nice-to-haves to v1.1
Misinterpretation as real medical tool	Low	Ethical/legal liability	Persistent disclaimers; synthetic data only; no real patient data ever

14. Appendix

14.1 Environment Variables

```
# .env
GOOGLE_API_KEY=<your-gemini-api-key>
GROQ_API_KEY=<your-groq-api-key>         # Optional fallback
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=<your-langsmith-key>
LANGCHAIN_PROJECT=medflow-ai
```

14.2 Python Dependencies

```
# requirements.txt
langchain>=0.3.0
langgraph>=0.3.0
langchain-google-genai>=4.0.0
langchain-groq>=0.2.0
langchain-community>=0.3.0
chromadb>=0.5.0
langsmith>=0.2.0
pydantic>=2.0
fastapi>=0.110.0
uvicorn>=0.27.0
python-dotenv>=1.0.0
requests>=2.31.0
```

14.3 Project File Structure

```
medflow-ai/
├── src/
│   ├── graph.py              # Top-level StateGraph (supervisor)
│   ├── state.py              # EncounterState, InputState, OutputState
│   ├── agents/
│   │   ├── intake.py         # Intake Agent sub-graph
│   │   ├── safety.py         # Safety Sentinel sub-graph
│   │   ├── evidence.py       # Evidence Synthesizer sub-graph
│   │   ├── triage.py         # Triage Router
│   │   ├── brief.py          # Clinical Brief Generator
│   │   └── education.py      # Patient Education Agent
│   ├── tools/
│   │   ├── fda_tools.py      # openFDA API tools
│   │   ├── pubmed_tools.py   # PubMed E-utilities tools
│   │   ├── rag_tools.py      # ChromaDB retriever tools
│   │   └── vision_tools.py   # Multimodal image analysis
│   ├── middleware/
│   │   ├── pii_redaction.py
│   │   ├── guardrails.py
│   │   └── disclaimers.py
│   ├── memory/
│   │   ├── store.py          # LangGraph Store setup
│   │   └── summarizer.py     # Conversation summarization
│   └── config.py             # LLM provider selection, assistants
├── scripts/
│   ├── ingest_medical_data.py   # ChromaDB data ingestion
│   └── run_evals.py             # LangSmith evaluation runner
├── frontend/                    # Next.js + React app
├── chroma_db/                   # Persistent ChromaDB storage
├── langgraph.json               # Deployment configuration
```

```
├── Dockerfile
├── requirements.txt
└── README.md
```

14.4 External API Reference

API	Endpoint	Auth / Rate Limit
openFDA Drug Labels	api.fda.gov/drug/label.json	Free API key, 240 req/min
openFDA Adverse Events	api.fda.gov/drug/event.json	Free API key, 240 req/min
RxNorm	rxnav.nlm.nih.gov/REST/	No auth, 20 req/sec
PubMed E-utilities	eutils.ncbi.nlm.nih.gov/entrez/eutils/	Free API key, 10 req/sec
NLM Clinical Tables	clinicaltables.nlm.nih.gov/api/	No auth, unlimited

14.5 Glossary

Term	Definition
ESI	Emergency Severity Index — 5-level triage classification (1=resuscitation, 5=non-urgent)
SOAP Note	Subjective, Objective, Assessment, Plan — standard clinical documentation format
RAG	Retrieval-Augmented Generation — grounding LLM responses in retrieved evidence
HITL	Human-in-the-Loop — requiring human approval before AI actions are finalized
MCP	Model Context Protocol — open standard for connecting LLMs to external tools
ChromaDB	Open-source vector database for embedding storage and similarity search
LangGraph Store	Persistent key-value store for long-term agent memory across sessions
Checkpointer	LangGraph mechanism that saves graph state after each node for persistence and time travel

---

E N D O F D O C U M E N T

MedFlow AI — PRD v1.0 — Confidential